For that reason, it's recommended to always combine the apt-get update and apt-get install commands into a single RUN instruction like so:[3]

```
RUN apt-get update -yqq && \
  apt-get install -yqq --no-install-recommends nodejs vim
```

This ensures that whenever you change the packages being installed, you'll also get the latest repository information at the same time.

Finally, it's good practice to format the apt-get install command as follows:

```
RUN apt-get update -yqq && apt-get install -yqq --no-install-recommends \
  nodejs \
  vim
```

Using one package per line and keeping packages in alphabetical order makes it easier to see which packages are installed, and locate ones that need to be changed if you have many packages installed.

Let's fix this issue in our Dockerfile now. We don't actually need Vim installed currently, so our two RUN instructions for apt-get update and apt-get install will become:

```
RUN apt-get update -yqq && apt-get install -yqq --no-install-recommends \
  nodejs
```

Let's rebuild our image to include this change:

```
$ docker build -t railsapp .
Sending build context to Docker daemon  102.9kB
Step 1/6 : FROM ruby:2.6
 ---> f28a9e1d0449
«...»
Successfully built 621ceaca3298
Successfully tagged railsapp:latest
```

Caching Issue 2: Unnecessary Gem Installs

Imagine that we want to make a change to our README.md file. Open this file in an editor, and replace the Rails default version with the following:

```
# README

This is a sample Rails application from Docker for Rails Developers (PragProg).
It was generated using Docker without Ruby installed on the local machine.

We're using the app to discover the wonderful world of Rails with Docker.
```

Now let's try something. What happens if we rebuild our image:

3. https://docs.docker.com/develop/develop-images/dockerfile_best-practices/#run